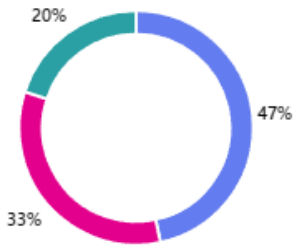
Föreläsning – Exceptions

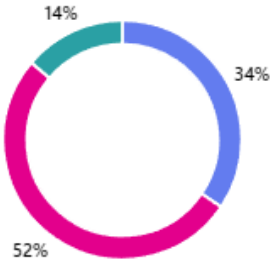
28. try, catch, finally

Jag förstår detta bra	14
Jag har delvis koll, men behöver öva mer	10
Jag behöver läsa på/förstå bättre	6



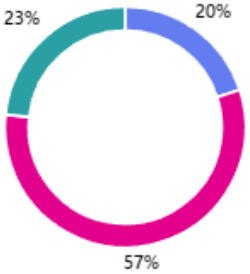
29. throw

Jag förstår detta bra	10
Jag har delvis koll, men behöver öva mer	15
Jag behöver läsa på/förstå bättre	4



30. När man bör hantera fel

Jag förstår detta bra	6
Jag har delvis koll, men behöver öva mer	17
Jag behöver läsa på/förstå bättre	7



C# Exceptions

När man kör C#-kod kan olika fel uppstå: kodningsfel som programmeraren gjort, fel på grund av felaktig input eller andra oförutsedda saker.

När ett fel uppstår kommer C# normalt att stoppa och generera ett felmeddelande. Den tekniska termen för detta är: C# kommer att kasta ett undantag (kasta ett fel).

C# try och catch

Try-satsen låter oss definiera ett kodblock som ska testas för fel medan det körs.

Catch-satsen låter dig definiera ett kodblock som körs om ett fel uppstår i try-blocket.

Try- och catch-nyckelorden kommer i par:

Så här ser syntaxen ut:

```
try
{
  //  Block of code to try
}
catch (Exception e)
{
  //  Block of code to handle errors
}
```

try

Omsluter kod som kan orsaka ett undantag (t.ex. fil-läsning, konvertering, nätverksanrop, databasanrop).

Programmet fortsätter köra som vanligt så länge inget fel inträffar här.

catch

Fångar ett specifikt undantag som uppstod i **try** och gör att programmet **inte kraschar.**

- Man kan ha flera **catch-block**, från mer specifika till generella **FormatException**, **Exception**).
- Man kan logga, visa meddelande, åtgärda problemet eller kasta ett nytt/globalt undantag.

finally

Kod här körs alltid, oavsett om det blev undantag eller inte.

- Perfekt för att stänga filer, databasanslutningar, streams etc.
- Används ofta istället för att duplicera "stäng allt" i både **try** och varje **catch**.

Finally-satsen låter oss köra kod efter try...catch, **oavsett resultat:**

Syntax

```
try
{
  int[] myNumbers = {1, 2, 3};
  Console.WriteLine(myNumbers[10]);
}
catch (Exception e)
{
  Console.WriteLine("Something went wrong.");
}
finally
{
  Console.WriteLine("The 'try catch' is finished.");
}
```

Throw

Vad throw gör och när man använder det

throw new ArgumentException(...)

Används när man själv vill signalera ett fel i sin kod, t.ex. om **argument är fel** eller logiken bryter mot kontrakt. Det säger: **"det här är inte ett normalt tillstånd, vi kan inte fortsätta"**.

Med throw satser kan man själv skapa anpassade felmeddelanden.

Throw används tillsammans med en **undantagsklass**.
Det finns många undantagsklasser tillgängliga i C#:
ArithmeticException, FileNotFoundException, IndexOutOfRangeException, TimeOutException, etc:

```
static void checkAge(int age)
{
  if (age < 18)
  {
    throw new ArithmeticException("Access denied - You must be at least 18 years old.");
  }
  else
  {
    Console.WriteLine("Access granted - You are old enough!");
  }
}

static void Main(string[] args)
{
  checkAge(15);
}
```

Får man detta errormeddelande

```
System.ArithmeticException: 'Access denied - You must be at least 18 years old.'
```

Skillnad mellan Throw new ... och bara Throw?

Throw new...

Skapar och kastar ett **nytt** undantagsobjekt.

Används när du **upptäcker ett fel** och vill markera det som ett undantag.

Throw utan något efter

Får bara användas **inuti ett catch-block**.

Kastar om **samma undantag** som just fångats, vidare i anropskedjan.

Används när du vill **logga eller göra något lokalt**, men låta någon högre upp hantera felet.

Exempel:

```
catch (FormatException ex)
{
    Console.WriteLine($"Formatfel: {ex.Message}");
    throw; // Skicka samma fel vidare
}
```

Vi loggade felet, men jag vill inte hantera det här – låt anroparen göra det.

Här fångas ett fel, loggas, och skickas vidare oförändrat till Main.

När man ska använda det

Undvik att fånga `Exception` och bara logga utan någon meningsfull åtgärd , det döljer ofta fel som borde ha letts vidare.

Hur skiljer sig produktionskod från utvecklingskod angående exceptions

I **produktionskod** hanterar man exceptions mer med **fokus** på **stabilitet**, **felsäkerhet** och **tydlig rapportering**, medan **utvecklingskod** ofta är mer **direkt** och **experimentell**.

I produktionskod vill man normalt undvika att "bara låta det krascha", men man vill också undvika att dölja fel med för bred catch.

Skillnaden i praktiken

I **utvecklingskod** är det vanligt att:

- snabbt fånga fel för att förstå vad som händer,
- använda Console.WriteLine eller debug-utskrifter,
- låta exceptions bubbla upp utan så mycket struktur.

I **produktionskod** vill man oftare:

- logga felet på ett strukturerat sätt,
- översätta tekniska fel till rätt API-svar eller användarmeddelande, vid backend-lösningar
- skilja på förväntade fel och oväntade fel,
- inte använda exceptions som vanlig kontrollflödeslogik.

Bra tumregler

- Fånga bara exceptions där du faktiskt kan göra något meningsfullt åt dem.
- Logga i produktionskod, men visa inte onödigt tekniska detaljer för användaren.

Utvecklingskod:

```csharp
try
{
    var product = service.GetProduct(id);
}
catch (Exception ex)
{
    Console.WriteLine(ex);
}
```

Produktionskod:

```csharp
try
{
    var product = service.GetProduct(id);
}
catch (ProductNotFoundException ex)
{
    return NotFound(ex.Message);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Oväntat fel vid hämtning av produkt {ProductId}", id);
    return StatusCode(500, "Ett oväntat fel inträffade.");
}
```

Några kontrollfrågor

Vad är skillnaden mellan throw new ArgumentException(...) och throw;?

Varför är finally användbart när man arbetar med filer?

Länkar

https://www.w3schools.com/cs/cs_exceptions.php

https://learn.microsoft.com/sv-se/dotnet/csharp/fundamentals/exceptions/how-to-handle-an-exception-using-try-catch

Exception-handling statements - throw and try, catch, finally - C# reference | Microsoft Learn